体系结构 exp10-exp11 实验报告

箱子：22

组员：叶驰骏 区梓俊 李皓

一、实验概述

本实验在 exp9 五级流水线和数据前递的基础上继续扩展 LoongArch 处理器的指令集。exp10 主要加入算术逻辑、移位、立即数、高位 PC 加法以及乘除法指令；exp11 在 exp10 基础上加入有符号/无符号条件分支和字节、半字访存指令。

处理器仍采用 IF、ID、EX、MEM、WB 五级流水线。每一级内部保存本阶段的流水寄存器，mycpu_top.v 主要负责阶段间连线和外部 SRAM 接口连接。对于不能在一个周期完成的除法运算，EX 阶段通过 allow_in 暂停 ID 和 IF；普通 ALU 结果继续通过前递路径减少停顿。

二、总体结构

流水级	职责	与其他部分的关系
IF 取指	取指令，管理 PC 和 IF/ID 寄存器	受 ID 的分支重定向和阻塞信号控制
ID 译码	指令译码、读寄存器、判断分支、检测相关	把操作数送给 EX；对 IF 发出重定向或阻塞
EX 执行	ALU 运算、乘除法、访存地址计算	ALU 结果前递回 ID；结果送 EX/MEM
MEM 访存	访问数据 SRAM，产生字节写使能	ALU 结果前递回 ID；数据送 MEM/WB
WB 写回	load 数据提取与扩展、写回寄存器堆	同周期写回结果旁路给 ID

图 1 五级流水结构与前递、阻塞关系

主要模块及职责如下：

模块	功能
if_stage.v	PC 管理、指令 SRAM 请求、IF/ID 寄存器和分支重定向
id_stage.v	指令译码、立即数生成、寄存器堆访问、分支比较、相关检测
ex_stage.v	ALU 运算、乘除法控制、访存地址计算、EX/MEM 寄存器
mem_stage.v	数据 SRAM 访问、字节写使能、MEM/WB 寄存器
wb_stage.v	load 数据提取和扩展、寄存器写回、调试输出
alu.v	加减法、比较、逻辑、移位和乘法组合逻辑
tools.v	独热译码器和 32 次迭代除法器
mycpu_top.v	五个阶段的实例化及信号连接

三、exp10：算术逻辑和乘除法指令

3.1 新增指令

exp10 新增以下指令：

- 立即数算术逻辑：slti、sltui、andi、ori、xori；
- 寄存器移位：sll.w、srl.w、sra.w；
- PC 相对高位加法：pcaddu12i；
- 乘法：mul.w、mulh.w、mulh.wu；
- 除法和取余：div.w、mod.w、div.wu、mod.wu。

这些指令在 id_stage.v 中根据 op_31_26_d 、op_25_22_d 、op_21_20_d 和 op_19_15_d 译码。译码结果组织为 19 位独热式 alu_op，低 12 位用于原有 ALU 运算和移位，[14:12] 表示三类乘法，[18:15] 表示四类除法/取余。这样可以让同一类运算共用

ALU 数据通路，同时由 EX 阶段单独处理多周期除法。

3.2 立即数和 PC 相对运算

slti 使用符号扩展的 si12，sltui 使用相同的 32 位扩展结果进行无符号比较；andi、ori 和 xori 的 ui12 采用零扩展。pcaddu12i 将 i20 符号扩展后左移 12 位，再与当前指令 PC 相加。ID 阶段通过 src1_is_pc 和 src2_is_imm 选择 PC、寄存器或立即数作为 ALU 输入。

3.3 乘法实现

alu.v 使用 Verilog 的乘法运算符推导乘法器。为了同时得到有符号和无符号乘法的高位结果，32 位操作数先扩展为 33 位有符号数：有符号数复制原符号位，无符号数最高位补 0。两个 33 位数相乘得到 66 位结果：

```
wire signed [32:0] signed_src1_33 = {alu_src1[31], alu_src1};
wire signed [32:0] signed_src2_33 = {alu_src2[31], alu_src2};
wire signed [65:0] signed_product_66 = signed_src1_33 * signed_src2_33;
```

mul.w 取低 32 位，mulh.w 取有符号乘积的 [63:32]，mulh.wu 取两个补零扩展操作数乘积的 [63:32]。在 Artix-7 目标器件上，Vivado 可将乘法推导到 DSP48 资源中；本设计没有在乘法器输出增加额外流水寄存器，因此乘法结果仍作为 EX 阶段的组合结果使用。

3.4 迭代除法器

tools.v 中的 divider 采用恢复除法算法，每个时钟周期生成一位商，共进行 32 次迭代。启动时保存被除数、除数绝对值和符号信息，运算期间更新 quotient_work、remainder_work 和计数器。第 32 次迭代完成后输出商和余数，并拉高 complete 一个周期。

除数为 0 时，除法器直接输出全 1 商和原被除数余数。is_signed 决定输入是否先取绝对值以及最终结果是否恢复符号；div.w、mod.w 使用有符号模式，div.wu、mod.wu 使用无符号模式。

EX 阶段用 div_started 防止同一条除法指令重复启动，并用 allow_in = !id_ex_valid || !is_div || (div_started && div_complete) 表示当前 ID/EX 内容何时可以进入

EX/MEM。除法未完成时，ID/IF 保持前端状态，EX/MEM 不接收无效的中间结果；完成后才把商或余数送入后续流水级。

3.5 exp10 中的相关处理

乘法结果属于普通 EX 结果，可以沿用 exp9 的 EX→ID、EX/MEM→ID 和 WB 同周期写回旁路。除法结果在 complete 前不可用，因此 EX 阶段将其目的寄存器作为 pending 结果反馈给 ID，相关指令暂时不能进入 ID/EX。

阻塞时的动作是：

1. IF 保持 PC 和 IF/ID 内容；
2. ID/EX 注入气泡或保持除法指令；
3. EX、MEM、WB 继续推进已经存在的指令；
4. 结果可用后，前端恢复推进。

3.6 exp10 波形图分析

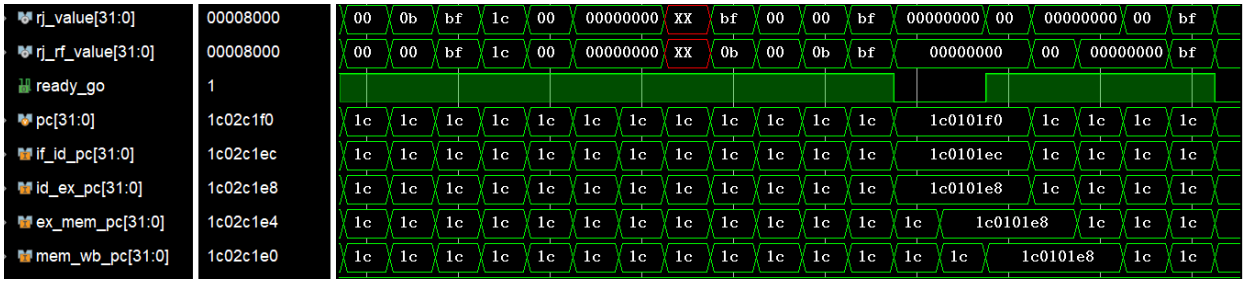
ready_go 表示当前 ID 级是否可以离开译码级、允许 IF 接收新指令。它是高电平有效：ready_go 为高时前端正常取指，为低时才进入阻塞。

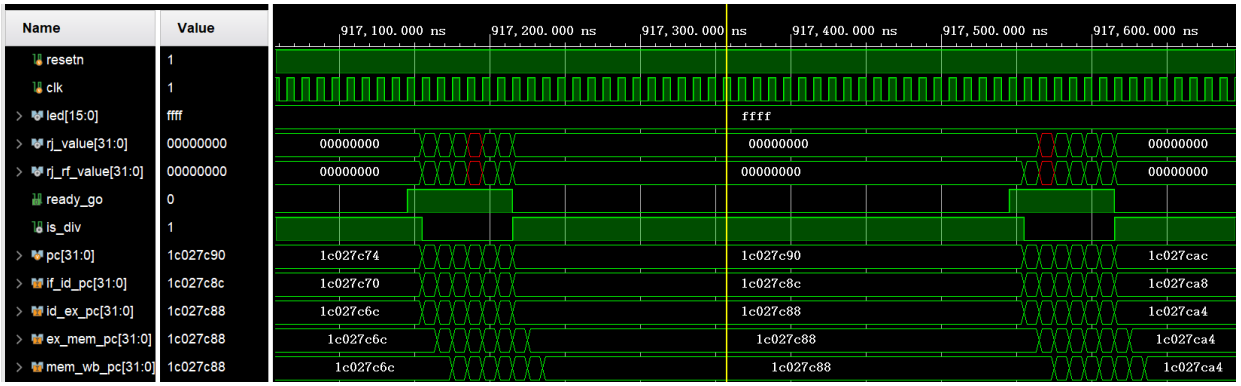
除法指令停顿 32 个周期

除法是 32 位恢复余数除法器，每拍产生 1 位商。start 拉高后，除法器内部 count = 32。

每拍 count 减 1，共迭代 32 次。count == 1 的那一拍把 complete 拉高，输出商和余数。

因此除法指令进入 EX 后，大约要让流水线停顿 32 拍，直到 div_complete 有效才继续。





四、exp11：分支和窄宽度访存

4.1 新增条件分支

exp11 新增：

- blt：有符号小于则跳转；
- bge：有符号大于等于则跳转；
- bltu：无符号小于则跳转；
- bgeu：无符号大于等于则跳转。

ID 阶段使用前递后的 rj_value 和 rkd_value 比较，分别生成：

```
wire rj_lt_rd  = $signed(rj_value) < $signed(rkd_value);
wire rj_ltu_rd = rj_value < rkd_value;
```

分支目标仍由当前 PC 加上符号扩展并左移两位的 i16 偏移得到。分支指令使用 rd 作为第二源寄存器，因此必须同时加入 use_rkd 和前递/阻塞检测。只有 if_id_valid、ready_go 和复位状态均有效时，br_taken 才能重定向 IF，避免气泡或阻塞状态产生错误跳转。

4.2 新增访存指令

exp11 新增：

指令	访问宽度	写回扩展
ld.b	1 字节	符号扩展
ld.h	2 字节	符号扩展
ld.bu	1 字节	零扩展
ld.hu	2 字节	零扩展
st.b	1 字节	不写回寄存器
st.h	2 字节	不写回寄存器

ID 阶段新增 mem_size 和 load_unsigned 控制位，并将它们依次保存到 ID/EX、EX/MEM 和 MEM/WB。mem_size=00 表示字节，01 表示半字，10 表示字。

4.3 store 的字节写使能

数据 SRAM 为 32 位数据总线并提供 4 位字节写使能。mem_stage.v 根据地址低两位选择写入掩码：

- st.b 只打开 4'b0001 << addr[1:0] 对应的一个字节；
- st.h 根据 addr[1] 打开低半字 0011 或高半字 1100；
- st.w 打开全部四个字节 1111。

4.4 load 的提取和扩展

RAM 返回一个对齐的 32 位字，WB 阶段依据保存的地址低两位提取目标字节或半字：

- 字节地址 00、01、10、11 分别选择 [7:0]、[15:8]、[23:16]、[31:24]；
- 半字根据地址最高的低位选择 [15:0] 或 [31:16]；
- 有符号 load 使用符号扩展；
- 无符号 load 使用零扩展；
- ld.w 直接使用完整 32 位返回值。

五、流水线接口和控制信号

阶段间的关键控制信号如下：

信号	作用
valid	标识流水级内容是否为真实指令，防止气泡产生写回或访存副作用
ready_go	ID 当前指令是否已经解决相关并可以离开 ID
allow_in	下一级是否能接收当前流水级内容
ex_pending_*	EX 中的 load/未完成除法目的寄存器，要求 ID 暂停
mem_pending_*	MEM 中尚未完成写回的 load 目的寄存器
ex_forward_*	EX 当前可直接前递的 ALU 或已完成除法结果
mem_forward_*	EX/MEM 中可直接前递的 ALU 结果
rf_we/rf_wdata	WB 实际写回端口，同时用于 ID 同周期旁路

前递选择优先级为 EX、MEM、WB、寄存器堆原始读值。分支比较、跳转基址和 store 数据都使用前递后的操作数，而不是只对 ALU 输入做前递。

六、问题分析

6.1 多周期除法的流水线停顿

除法器不是一周期组合运算，若让后续指令直接经过，会读到尚未完成的结果。div_started、complete 和 allow_in 共同保证除法指令在 EX 中保持，完成后才进入 EX/MEM。相关检测还要把未完成除法目的寄存器作为 pending 结果反馈给 ID。

6.2 窄宽度访问的地址对齐

字节访问使用地址低两位选择四个 lane；半字访问使用 addr[1] 选择低半字或高半字，addr[0] 应为 0。store 需要配合 byte-enable，load 则需要在 WB 重新选择并扩展。访存宽度、符号属性和地址低位必须作为控制信息随流水线同步传递。

6.3 分支相关和数据前递

新分支使用 rj 和 rd 比较。如果比较操作数不使用前递，紧邻 ALU 指令或 load 后的分支会使用旧值。设计将分支加入 use_rj/use_rkd，并复用已有的 EX/MEM/WB 前递和 load-use 阻塞机制。

八、实验总结

exp10 完成了从基础整数 ALU 到乘除法单元的扩展：立即数和移位指令复用了原有 ALU，乘法利用 33 位符号扩展统一处理有符号/无符号高位结果，除法通过 32 次迭代并用 EX 停顿机制接入五级流水线。exp11 进一步把分支比较和窄宽度访存接入已有数据通路，利用 mem_size、load_unsigned 和 4 位 byte-enable 实现字节、半字 load/store。

这两次实验说明，扩展指令不仅需要增加译码和运算逻辑，还必须同步扩展流水寄存器、前递、相关检测、访存控制和调试接口。

九、调试过程与真实体会

这次最花时间的不是加法器或乘法器，而是几根控制信号的极性和时序。比如 ready_go

和 allow_in，名字上都像“能不能继续”，一开始我总把高电平理解成阻塞，直到在波形里把 ready_go 与 PC、if_id_valid 叠在一起，看到 ready_go 拉低时前端才冻结，才彻底分清高电平表示允许。load-use 停顿表面上看只插入了一个气泡，实际要连看 ex_pending_we 和 mem_pending_we，才知道 load 在 EX 和 MEM 各停一拍，总共是两拍。

除法我一开始图省事直接写了 a / b，综合时 Vivado 确实把它映射到了 DSP，但后续时序报告里 WNS 为负。这让我意识到，写成一行能过功能仿真，不等于能过时序。后来才认真考虑是否要加输入输出流水寄存器，以及 DSP 的使能方式。

除法从组合逻辑改成 32 拍迭代后，功能正确了，但每条 div、mod 都要占住 EX 约 32 拍。当时我觉得停顿太长，后来对照参考实现和教材，发现除法结果前递的简单方案基本就是这个数量级。实验里“能综合出结果”和“功能正确、时序收敛、板上稳定”是三件不同的事，这是这次实验最大的收获。